

The Windows® 95 User Interface: A Case Study in Usability Engineering

Kent Sullivan
Microsoft Corporation
One Microsoft Way
Redmond, WA 98052-6399
+1 206 936 3568
kentsu@microsoft.com

ABSTRACT

The development of the user interface for a large commercial software product like Microsoft® Windows 95 involves many people, broad design goals, and an aggressive work schedule. This design briefing describes how the usability engineering principles of iterative design and problem tracking were successfully applied to make the development of the UI more manageable. Specific design problems and their solutions are also discussed.

Keywords

Iterative design, Microsoft Windows, problem tracking, rapid prototyping, usability engineering, usability testing.

INTRODUCTION

Windows 95 is a comprehensive upgrade to the Windows 3.1 and Windows for Workgroups 3.11 products. Many changes have been made in almost every area of Windows, with the user interface being no exception. This paper discusses the design team, its goals and process then explains how usability engineering principles such as iterative design and problem tracking were applied to the project, using specific design problems and their solutions as examples.

Design Team

The Windows 95 user interface design team was formed in October, 1992 during the early stages of the project. I joined the team as an adjunct member, to provide usability services, in December 1992. The design team was truly interdisciplinary, with people trained in product design, graphic design, usability testing, and computer science. The number of people oscillated during the project but was approximately twelve. The software developers dedicated to implementing the user interface accounted for another twelve or so people.

Design Goals

The design team was chartered with two very broad goals:

- Make Windows easier to *learn* for people just getting started with computers and Windows.

Permission to make digital/hard copies of all or part of this material for personal or classroom use is granted without fee provided that the copies are not made or distributed for profit or commercial advantage, the copyright notice, the title of the publication and its date appear, and notice is given that copyright is by permission of the ACM, Inc. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires specific permission and/or fee.

CHI 96 Vancouver, BC Canada

© 1996 ACM 0-89791-777-4/96/04..\$3.50

- Make Windows easier to *use* for people who already use computers—both the typical Windows 3.1 user and the advanced, or “power”, Windows 3.1 user.

With over 50 million units of Windows 3.1 and 3.11 installed plus a largely-untapped home market, it was clear from the outset that the task of making a better product was not going to be a trivial exercise. Without careful design and testing, we were likely to make a product that improved usability for some users and worsened it for millions of other users (existing or potential). We understood fairly well the problems that intermediate and advanced users had but we knew little about problems beginning users had.

Design Process

Given very broad design goals and an aggressive schedule for shipping the product (approximately 18 months to design and code the user interface) we knew from the outset that a traditional “waterfall” style development process would not allow us sufficient flexibility to attain the best-possible solution. In fact, we were concerned that the traditional approach would yield a very unusable system.

In the “waterfall” approach, the design of the system is compartmentalized (usually limited to a specification writing phase) and usability testing typically occurs near the end of the process, during quality assurance activities. We recognized that we needed much more opportunity to create a design, try it out with users (perhaps comparing it to other designs), make changes, and gather more user feedback. Our desire to abandon the waterfall model and opt for iterative design fortunately followed similar efforts in other areas of the company, so we had concrete examples of its benefits and feasibility.

ITERATIVE DESIGN IN PRACTICE

Figure 1 outlines the process that we used. The process was typical of most products designed iteratively: paper or computer-based prototypes were used to try out design ideas and to gather usability data in the lab. Once a design had been coded, it was refined in the usability lab. When enough of the product had been coded and refined, it was examined more broadly, over time, in the field. Minor usability problems identified in the field were fixed before shipping the product. More importantly, the data gathered in the field is being used to guide work on the next version.

Our iterative design process was divided into three major phases: exploration, rapid prototyping, and fine tuning.

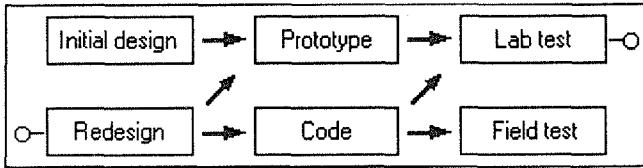


Figure 1: Windows 95 iterative design process.

Exploration Phase

In this first phase we experimented with design directions and gathered initial user data. We began with a solid foundation for the visual design of the user interface by leveraging work done by the "Cairo"¹ team. We inherited from them much of the fundamental UI and interaction design (the desktop, the "Tray", context menus, three-dimensional look and feel, etc.). We also collected data from product support about users' top twenty problems with Windows 3.1.

Figure 2 shows a prototype Windows 95 desktop design that we usability tested in January 1993². This design was based on Cairo and incorporated a first pass at fixing some of the known problems with Windows 3.1 (window management in particular).

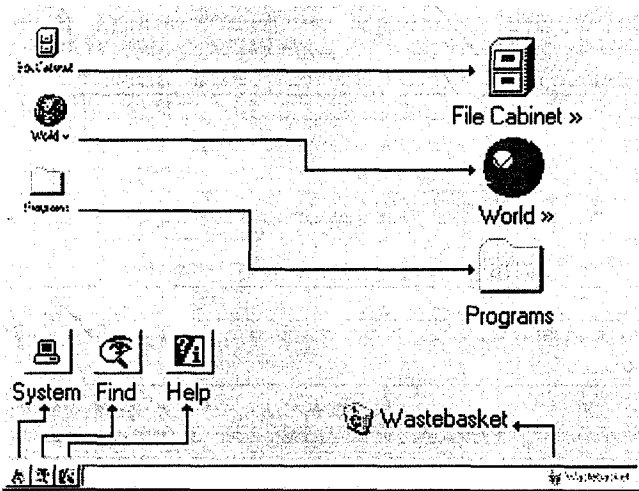


Figure 2: Early Windows 95 desktop (with callouts to enhance clarity).

The top icon, File Cabinet, showed a Windows 3.1 File Manager-type view (left pane shows hierarchy, right pane show contents). The second icon, World, showed items on the network. The third icon, Programs, was a folder which contained other folders full of links to programs on the computer. Along the bottom was the "Tray", which featured three buttons (System, Find, and Help) and a file storage area. Another icon, Wastebasket, was a container for deleted files.

¹ A future version of Microsoft Windows NTTM. Initial design work for the Cairo UI began in 1990.

² Using a mixture of paper, Microsoft Visual BasicTM, and code (C language).

The usability studies of the prototype desktop were conducted in the Microsoft usability lab, as were later tests.³ We conducted typical iterative usability studies. Three to four users representing each distinct group of interest (typically beginning and intermediate Windows 3.1 users) completed tasks which exercised the prototype. Questions we addressed in testing were sometimes very broad (e.g., "Do users like it?") and sometimes very specific (e.g., "After ten minutes of use, do users discover drag and drop to copy a file?"). We collected data typical for iterative studies: verbal protocols, time per task, number of errors, types of errors, and rating information.

Early Findings

Our usability testing of this prototype revealed much including several surprises:

- Beginning users and many intermediates were confused by the two-pane view of File Cabinet. (See Figure 3.) They were unsure of the relationship between the panes and how to navigate between folders. Beginners were often overwhelmed by the visual complexity of the File Cabinet and had more basic problems, such as not understanding how folders could exist inside of other folders. Many users were also confused by the Parent Folder icon. It appeared in every folder and looked like a file, yet was really a navigation control for moving up the hierarchy one level.

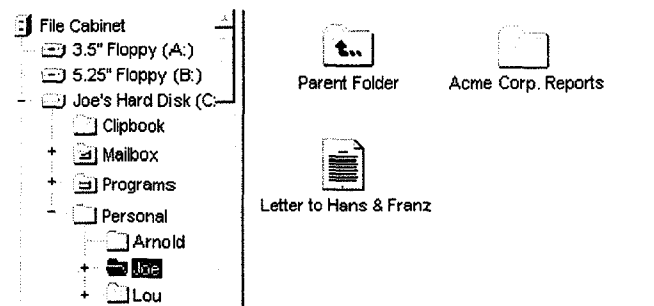


Figure 3: File Cabinet, an early file system viewer.

- Users of every type were confused by the Programs folder. We thought that having a folder on the desktop with other folders and links to programs inside it would be a natural transition for Windows 3.1 users accustomed to Program Manager, while being relatively easy to learn for beginners. We were wrong! Beginners quickly got lost in all of the folders (unlike File Cabinet, each folder opened into a different window) and other users had a lot of trouble deciding whether they were looking at the actual file system and its files or just links to actual files.
- Users had considerable difficulty deciding what each of the three buttons on the Tray was for and later had trouble remembering where to go for a particular command because their functions overlapped in certain

³ For a summary of the usability lab facilities, see chapter 11 of [5].

contexts (e.g., to find something in Help, do you go to Find or to Help?).

Comparison to Windows 3.1

From the first lab studies it became clear that we needed a baseline with Windows 3.1, to better understand what problems existed prior to Windows 95 and what problems were unique to the new design. First, we gathered market research data about Windows 3.1 users' twenty most-frequent tasks. We then conducted several lab studies comparing Windows 3.1 and Windows 95, focusing on the top twenty tasks derived from the market research data. We also interviewed professional Windows 3.1 (and Macintosh, for comparison) educators, to learn what they found easy and difficult to teach about the operating system.

The key findings were:

- In Windows 3.1, beginning users took over 9 ½ minutes, on average, to locate and open a program that was not immediately visible. Results were not much better for our Windows 95 prototype. These results were clearly unacceptable, given that our market research data (and common sense) told us that starting a program was users' number one task.
- Beginning users and some intermediates had a lot of trouble using the mouse, especially double-clicking. As a result, they often failed to find things in containers when the only way to open them was double-clicking.
- Beginning users and many intermediates relied almost exclusively on visible cues for finding commands. They relied on (and found intuitive) menu bars and tool bars, but did not use pop-up (or "context") menus, even after training.
- All but the most advanced users did not understand how to manage overlapping windows efficiently. Beginners had the most trouble—when they minimized a window, they considered it "gone" if it was obscured by another window. We heard many stories from educators (and witnessed in the lab) how users caused the computer to run out of RAM by starting multiple copies of a program instead of switching back to the first copy. Intermediate users were more proficient but still had trouble, especially with Multiple-Document-Interface (MDI) applications such as Program Manager and Microsoft Word. Market research data confirmed the problem by revealing that 40% of intermediate Windows users didn't run more than one program at a time because they had some kind of trouble with the process.⁴
- Beginning users were bewildered by the hierarchical file system. Intermediate users could get around in the hierarchy, but often just barely, and usually saved all

of their documents in the default directory for the program they were using. This problem (especially the novice case) was also observed with Macintosh users.

A Change of Direction

The results from these studies and interviews greatly changed the design of the Windows 95 UI. In the early Windows 95 prototype, we had purposefully changed some things from Windows 3.1 (e.g., the desktop was now a real container) but not others (e.g., File Manager and Program Manager-like icons on desktop) because we were afraid of going too far with the design. We were aware that creating a product which was radically different from Windows 3.1 could confuse and disappoint millions of existing users, which would clearly be unacceptable.

However, the data we collected with the Windows 95 prototype and with Windows 3.1 showed us that we couldn't continue down the current path. The results with beginning users on basic tasks were unacceptably poor and many intermediate users thought that Windows 95 was just different, not better.

We decided to step back and take a few days to think about the situation. The design team held an offsite retreat and reviewed all the data collected to date: baseline usability studies, interviews, market research, and product support information. As we discussed the data, we realized that we needed to focus on users' most-frequent tasks. We also realized that we had been focusing too much on consistency with Windows 3.1.

Essentially, we realized that a viable solution might not look or act like Windows 3.1 but would definitely provide enough value to be attractive for users of all levels, for potentially different reasons. We realized that a truly usable system would *scale* to the needs of different users: it would be easy to discover and learn yet would provide efficiency (through shortcuts and alternate methods) for more-experienced users.

Rapid Iteration Phase

As we started working on new designs, we hoped to avoid the classic "easy to learn but hard to use" paradox by always keeping in mind that the basic features of the UI must scale. To achieve this goal, we knew we needed to try many different ideas quickly, compare them, and iterate those which seemed most promising. To do this, we needed to make our design and evaluation processes very efficient.

UI Specification Process Evolution

Although we had opted for an iterative design approach from the beginning, one legacy of the waterfall design approach remained: the monolithic design specification ("spec"). During the first few months of the project, the spec had grown by leaps and bounds and reflected hundreds of person-hours of effort. However, due to the problems we found via user testing, the design documented in the spec was suddenly out of date. The team faced a major decision: spend weeks changing the spec to reflect the new ideas and lose valuable time for iterating or stop

⁴ Interestingly, educators told us this problem also existed on the Macintosh, even though the UI for switching among programs was totally different.

updating the spec and let the prototypes and code serve as a "living" spec.

After some debate, the team decided to take the latter approach. While this change made it somewhat more difficult for outside groups to keep track of what we were doing, it allowed us to iterate at top speed. The change also had an unexpected effect: it brought the whole team closer together because much of the spec existed in conversations and on white boards in people's offices. Many "hallway" conversations ensued and continued for the duration of the project.

To ensure that interested parties stayed informed about the design, we:

1. Held regular staff meetings for the design team. These weekly (sometimes more often) meetings allowed each of us to check in about what we were doing and to efficiently discuss how what one person was working on affected other work.
2. Broadcasted usability test schedules and results via electronic mail. Design team members received regular notification of upcoming usability tests and results from completed tests so they could more easily keep abreast of the usability information and how the design was evolving.
3. Formally tracked usability issues. With a project the size of Windows 95, we knew we needed a standard way to note all of the usability issues identified, record when and how they were to be fixed, and then close them once the fix was implemented and tested successfully with users. This process is discussed more in the "Keeping Track of Open Issues" section.
4. Held regular design presentations for outside groups. As the project progressed, more and more groups (inside and outside Microsoft) wanted to know what we were doing, so we showed them and demonstrated what we were working on. These presentations were more effective than a written document, because the

presentations were easier to keep up-to-date and allowed timely design discussions.

Separate UI for Beginners

The first major design direction we investigated was a separate UI ("shell") for beginning users. The design was quickly mocked up in Visual Basic and tested in the usability lab. (See Figure 4.) While the design tested well, because it successfully constrained user actions to a very small set, we quickly began to see the limitations as more users were tested:

1. If just one function a user needed was not supported in the beginner shell, s/he would have to abandon it (at least temporarily).
2. Assuming that most users would gain experience and want to leave the beginner shell eventually, the learning they had done would not necessarily transfer well to the standard shell.
3. The beginner shell was not at all like the programs users would run (word processors, spreadsheets, etc.). As a result, users had to learn two ways of interacting with the computer, which was confusing.

For these reasons and others, we abandoned the idea. Importantly, because we used a prototyping tool and tested immediately in the usability lab, we still had plenty of time to investigate other directions.

Rapid Iteration Examples

Below are overviews of five areas where we designed and tested three or more major design iterations. There are many more areas for which there is not adequate space to discuss.

1. Launching Programs: Start Menu. Although we abandoned the idea of a separate shell for beginners, we salvaged its most useful features: single-click access, high visibility, and menu-based interaction. We mocked up a number of representations in Visual Basic and tested them with users of all experience levels, not

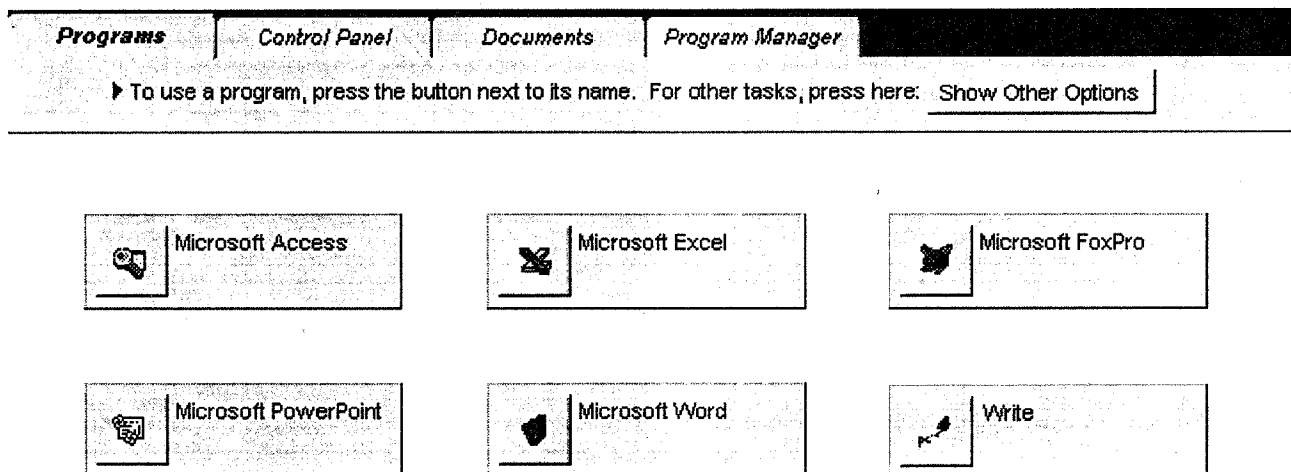


Figure 4: Partial view of separate shell for beginners.

just beginners, because we knew that the design solution would need to work well for users of varying experience levels. Figure 5 shows the final Start Menu, with the Programs sub-menu open. The final Start Menu integrated functions other than starting programs, to give users a single-button home base in the UI.

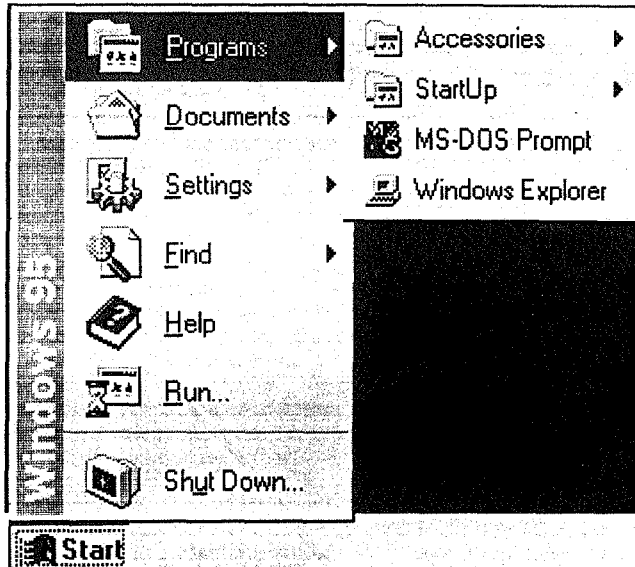


Figure 5: Start menu with Programs item open.

2. **Managing Windows: Task Bar.** Our first design idea for making window management easier was not very ambitious, but we weren't sure how much work was needed to solve the problem. The first design was to change the look of minimized windows from icons to "plates". (See Figure 6.) We hoped that the problem would be solved by giving minimized windows a distinctive look and by making them larger. We were wrong! Users had almost exactly the same amount of trouble as with Windows 3.1. Our testing data told us that the main problem was windows not being visible at all times, so users couldn't see what they had open or access tasks quickly. This realization led us fairly quickly to the task bar design, shown in Figure 9. Every task has its own entry in the task bar and the bar stays on top of other windows. User testing confirmed that this was a feasible solution to the problem.



Figure 6: "Plate" visualization for minimized windows.

3. **Working with Files: "Open" and "Save As" dialogs.** Information from product support plus lab testing told us that beginners and intermediates had a lot of trouble using the system-provided dialogs for opening and

saving files. (See Figure 7.) The problems stemmed from the fields in the dialog not being in a logical order and having a complex selection methodology. The Cairo team took the lead on this problem and constructed a comprehensive Visual Basic prototype that included a mock file system. We tested many variations until we arrived at the final design shown in Figure 8.

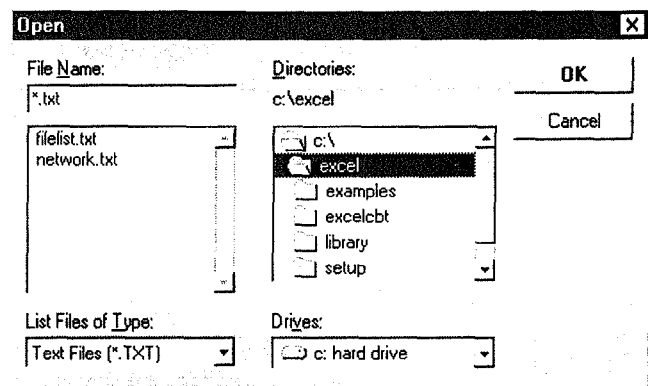


Figure 7: Windows 3.1 File Open dialog box.

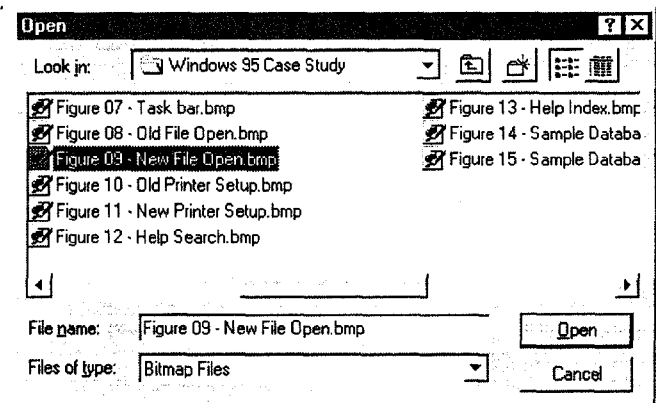


Figure 8: Windows 95 File Open dialog box.

4. **Printing: Setup Wizard.** Product support information told us that printer setup and configuration was the number one call-generator in Windows 3.1. Many of the problems stemmed from the printer setup UI. (See Figure 10.) Searching for a printer was difficult because all printers were in one long list. Choosing a port for the printer, especially in a networked environment, required tunneling down 4-5 levels and featured non-standard and complicated selection behavior. About the time we started work on this problem, members of the design team began investigating wizards as a solution to multi-step, infrequent tasks. Printer setup fit this definition nicely and the resulting wizard tested very well with users. The printer selection screen from the final wizard is shown in Figure 11.



Figure 9: Task bar with Start button, programs, and clock.

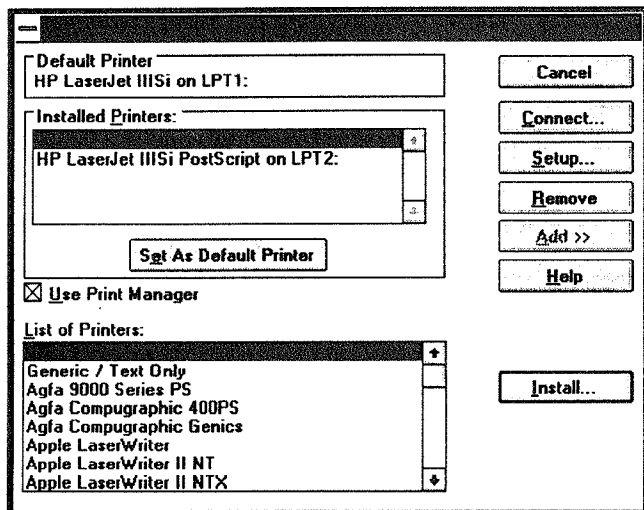


Figure 10: Main Windows 3.1 printer setup dialog box.

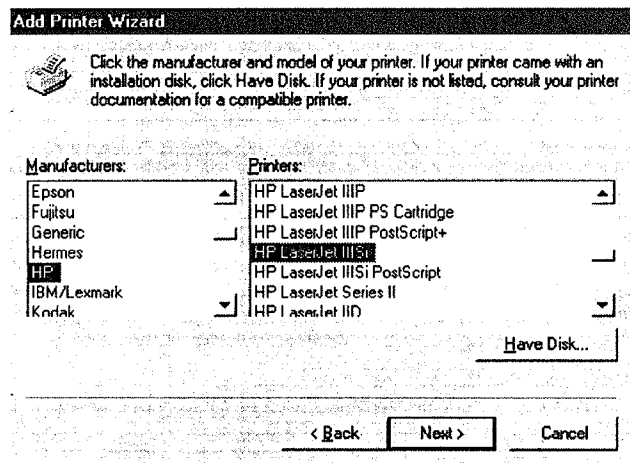


Figure 11: Screen from Windows 95 Add Printer wizard.

5. Getting Help: Search dialog/Index tab. Lab testing of Windows 3.1 showed that users had trouble with the Search dialog in Help. (See Figure 12.) Users had difficulty understanding that the dialog was essentially two parts and that they needed to choose something from the first list and then from the second list, using different buttons. We tried several ideas before arriving at the final Index tab. (See Figure 13.) The Index tab only has one list, and keywords with more than one topic generate a pop-up dialog that users have no trouble noticing.

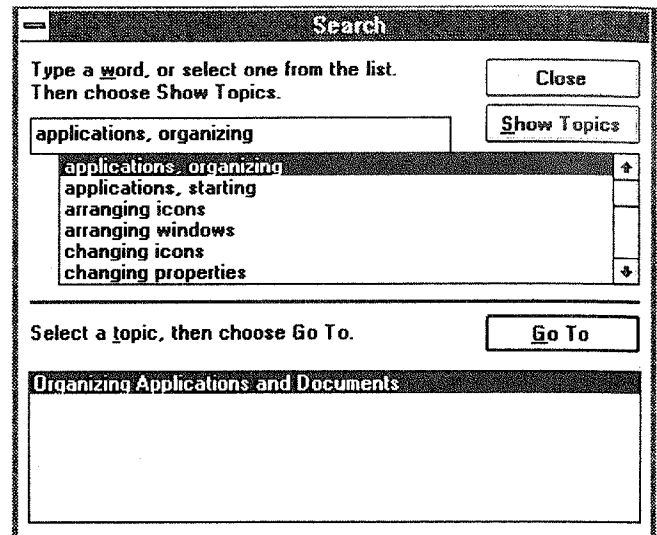


Figure 12: Windows 3.1 Help•Search dialog.

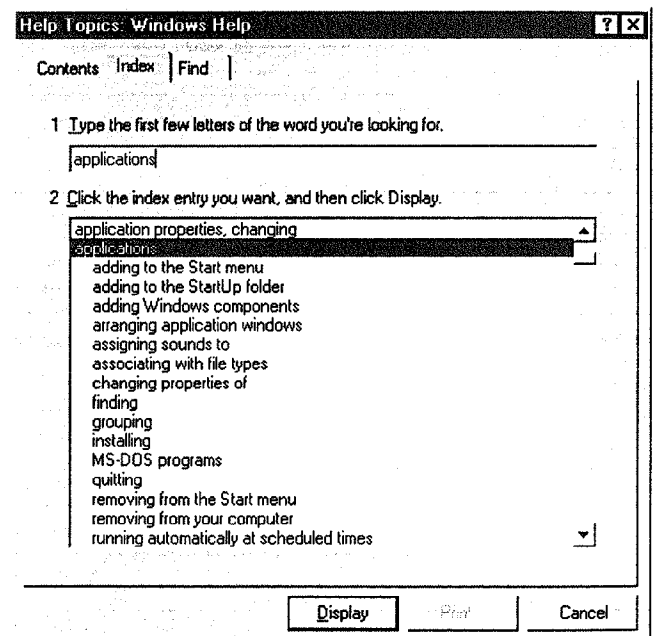


Figure 13: Windows 95 Help•Index tab.

Fine Tuning Phase

Once we had designed all of the major areas of the product, we realized that we had to take a step back and see how all of the pieces fit together. To accomplish this, we conducted summative lab tests and a longitudinal field study.

- Summative lab testing. Using the top twenty tasks identified from market research, we conducted holistic tests of the entire UI. Users of different experience levels completed isomorphic sets of tasks, to measure ease of learning and ease of use once learned. We compared performance with Windows 3.1 as a baseline. After piloting the test in-house to work out problems with the procedure, the test was conducted by an outside vendor, so that the results could be used in a white paper [3]. The results were very encouraging—users finished the tasks in about half the

time it took them in Windows 3.1 and they were more satisfied with Windows 95 in 20 of the 21 categories surveyed.

- **Longitudinal field study.** Using the final beta of Windows 95, we conducted a 20-person field study. We first examined how users worked with Windows 3.1 then watched them set up Windows 95. We returned after a week and after a month to measure learning and changes in use over time. We did not find any major usability holes in the product but did tweak wording in the UI and in Help topics. Some of the data collected is being used by product planners for the next version of Windows and also by product support, as a concise list of things to watch out for when taking support calls.

KEEPING TRACK OF OPEN ISSUES

Throughout the course of designing and testing the Windows 95 UI, we applied various usability engineering principles and practices [2] [4]. With a project the size of Windows 95, we knew we needed a standard way to note all of the usability issues identified, record when and how they were to be fixed, and then close them once the fix was implemented and tested successfully with users.

We designed a relational database to meet this need. (See Figure 14.) After every phase of lab testing, I entered new problems as well as positive findings and assigned them to the appropriate owners—usually a designer and a user education person together. The status of existing problems was also updated—either left open if more work was needed or closed if solved. Every couple of weeks I ran a series of reports that printed all of the remaining problems, by owner, and distributed them to the team members. (See Figure 15.) We met to discuss progress on solutions and when the changed designs would be ready to test with users.

Figure 14: Sample tracking database record.

Closed Usability Issues by UI Area		All Owners
06-Sep-95		1
Explorer		
Result or Problem:	Two subjects attempted to delete a file via Edit.Cut. They didn't realize that this hadn't actually deleted the file, just marked it for moving.	
Severity:	2	
Recommendations(s):	None.	
Addressed?:	Yes	
Description:	A cut file doesn't disappear but its icon dims to help indicate its new status.	
Test Information:	Chicago Shell Phase 26	
Related UI Area(s):	Transfer Model	
Result or Problem:	Some subjects tried to paste the copy of the file into the source folder then drag it to the destination. This action was not supported.	
Severity:	2	
Recommendations(s):	Allow a file to be pasted in the source folder. Rename it slightly to avoid conflicts.	
Addressed?:	Yes	
Description:		
Test Information:	Chicago Shell Phase 26	

Figure 15: Sample tracking database report.

Report Card

As with any project, the “proof is in the pudding” so sharing some summary statistics is in order.

Lab Testing

We conducted sixty-four phases of lab testing, using 560 subjects. Fifty percent of the users were intermediate Windows 3.1 users; the rest were beginners, advanced users, and users of other operating systems. These numbers do not include testing done on components delivered to us by other teams (Exchange email client, fax software, etc.) Testing on those components accounts for approximately 25 phases and 175 users.

Problem Identification

For the core shell components⁵, 699 different “usability statements” were entered into the database during the project. Of that number, 148 were positive findings and 551 were problems. The problems were rated with one of three levels of severity:⁶

- Level 1: Users were unable to continue with a task or series of tasks due to the problem.
- Level 2: Users had considerable difficulty completing a task or series of tasks but were eventually able to continue.
- Level 3: Users had minor difficulty completing a task or series of tasks.

Of the 551 problems identified, 15% were judged to be level 1, 43% level 2, and 42% level 3.

Problem Resolution

During the project, there were five types of resolution:

1. Addressed. The team fixed the problem and it tested successfully with users.

⁵ Does not include components delivered by other teams.

⁶ Based on a rating scale defined in [1].

2. Planned. The team designed a fix for the problem and we are waiting for it to be implemented.
3. Undecided. The team is not sure whether to fix the problem or is unsure if a fix is feasible.
4. Somewhat. The team designed a fix and it was tested with users, and the results were satisfactory but some issues remain.
5. Not Addressed. The team is not going to fix the problem.

By the end of the project, all problems with resolution "planned" or "undecided" had migrated to one of the other categories. Eighty-one percent of the problems were resolved "Addressed", 8% were resolved "Somewhat", and 11% were resolved "Not Addressed". Most of the issues that were not addressed were due to a technical limitation, or sometimes a scheduling limitation.

CONCLUSIONS

The Windows 95 project was the first experience for many of the team members for doing iterative design, usability testing, and problem tracking.

Iterative Design

Perhaps the best testament to our belief in iterative design is that literally no detail of the initial UI design for Windows 95 survived unchanged in the final product. At the beginning of the design process, we didn't envision the scope and volume of changes that we ended up making. Iterative design, using prototypes and the product as the spec, and our constant testing with users allowed us to explore many different solutions to problems quickly.

The design team became so used to iterating on a design that we felt rushed when, near the end of the project, we had to do some last-minute design work. There wasn't sufficient time to iterate more than once. We were disappointed that we didn't have time to continue fine tuning and re-testing the design.

Specification Process

The "prototype or code are the spec" approach overall worked well, although we naturally have refined the process over time. For example, all the prototypes for a given release of the product now reside in a common location on the network and include instructions for installing and running them.

The design team continues to write initial specification documents and circulate them for early feedback. Once prototyping and usability testing has begun, however, the spec often refers readers to the prototype for details. We have essentially found that the prototype is a richer type of specification, for less work, since it has other uses (usability testing, demos, etc.). A prototype also invites richer feedback, because the reviewer has to imagine less about how the system would work.

Usability Testing

Although doing design and user testing iteratively allowed us to create usable task areas or features of the product, user testing the product holistically was key to polishing the fit between the pieces. As discussed previously, we made changes to wording in the UI and in Help topics based on the data collected. If we had not done this testing, users' overall experience with the product would have been less productive and enjoyable.

Problem Tracking

The high fix rate for usability problems would not have been possible without the intense dedication of all the team members. The tracking database made the whole process more manageable and ensured that issues didn't slip between the cracks. However, the fixes would not have been made if the team had not believed in making the most-usable product possible. Key to this belief was our understanding that we probably weren't going to get it right the first time and that *not* getting it right was as useful and interesting to creating a product as getting it right was.

In the tracking database, all of the issues marked "Somewhat" or "Not Addressed" were rolled over into a new database, as a starting point for design work on the next version of Windows. Product planners and designers worked with the information on a daily basis, as well as processing reports from product support.

ACKNOWLEDGMENTS

Thanks to Jane Dailey, Chris Guzak, Francis Hogle, Marshall McClintock, Mark Malamud, Suzan Marashi, and Mark Simpson for reviewing this design briefing and providing comments. Thanks to Lauren Gallagher, Shawna Sandeno, and Jennifer Shetterly for graphic design assistance.

REFERENCES

1. Dumas, J. S. and Redish, J. C. (1993). *A Practical Guide to Usability Testing* (pp. 324-325). Norwood, NJ: Ablex Publishing Company.
2. Nielsen, J. (1993). *Usability Engineering*. San Diego, CA: Academic Press, Inc.
3. Usability Sciences Corporation. (1994). *Windows 3.1 and Windows 95 Quantification of Learning Time & Productivity*. (Available from <http://www.microsoft.com/windows/product/usability.htm>.)
4. Whiteside, J. L., Bennett, J., & Holtzblatt, K. (1988). Usability Engineering: Our Experience and Evolution. In M. Helander (Ed.), *Handbook of Human-Computer Interaction* (pp. 791-817). Amsterdam: Elsevier Science Publishers, B. V.
5. Wiklund, M. E. (1994). *Usability in Practice: How Companies Develop User-Friendly Products*. Cambridge, MA: Academic Press, Inc.